

Post-Quantum Cryptography in the Era of AI Agents: Securing Distributed Model Context Protocols

Yoshihiro Ando
Independent Researcher
yosh5295@gmail.com
Tokyo, Japan

Abstract—As autonomous AI agents transition from isolated tools to interconnected entities via the Model Context Protocol (MCP), securing their identity and audit trails against quantum threats becomes a critical imperative for zero-trust governance. This paper proposes a Post-Quantum Cryptographic (PQC) framework for AI agents, featuring a real-world implementation using NIST-standardized ML-DSA-65 (FIPS 204). Through benchmarking within the `llm-cli` reference implementation, we analyze the trade-offs between cryptographic resilience and operational efficiency. We demonstrate that while current pure-Python implementations introduce a signing latency of approximately 92 ms, this overhead remains negligible compared to the inference cycles of Large Language Models (LLMs). Furthermore, we address the critical challenge of *Context Exhaustion* caused by massive PQC signatures (inflating payloads by $9\times$ – $15\times$), and propose a context-aware “Signature Stripping” mechanism essential for maintaining long-term agent reasoning capability. By discussing scalability to ML-DSA-87 (AES-256 equivalent) for national-scale infrastructure, this work provides a foundational roadmap for building quantum-safe autonomous systems that are secure, compliant, and practically deployable.

Index Terms—Artificial Intelligence, Autonomous Agents, Model Context Protocol, Post-Quantum Cryptography, ML-DSA, Zero Trust Architecture, Cybersecurity

I. INTRODUCTION

The rapid advancement of Large Language Models (LLMs) has catalyzed the transition from passive text generators to proactive, autonomous AI agents capable of reasoning, planning, and executing complex workflows. To enable interoperability among distributed tools, agents, and remote environments, standardized communication frameworks such as the Model Context Protocol (MCP) have emerged as essential infrastructure. In robust zero-trust implementations, such as the `llm-cli` framework, MCP facilitates secure orchestration by employing asymmetric identity propagation, workload-bound tokens (JWTs), and chained-hash audit logging governed by classical public-key cryptography, notably RS256.

While these Zero Trust architectures successfully mitigate classical threats such as prompt injection and unauthorized tool execution, they remain fundamentally vulnerable to the looming threat of quantum computing. Shor’s algorithm, executable on future fault-tolerant quantum computers, can efficiently solve the integer factorization and discrete logarithm problems, rendering current RSA and Elliptic Curve Cryptography (ECC) obsolete.

In an interconnected agentic ecosystem, compromised cryptography implies catastrophic vulnerabilities. Adversaries could spoof agent identities to bypass Role-Based Access Controls (RBAC), inject malicious context, or forge cryptographically signed audit logs to conceal unauthorized actions. This is further exacerbated by “Store Now, Decrypt Later” (SNDL) attacks, where current agent communications are harvested to be exploited once quantum capabilities mature.

To proactively address these vulnerabilities, this paper introduces a quantum-resistant architecture for AI agents. We investigate the feasibility of integrating Post-Quantum Cryptography (PQC)—specifically the NIST-standardized lattice-based signature scheme ML-DSA-65 (Dilithium3)—into the distributed agent workflow.

II. BACKGROUND AND THREAT MODEL

A. Autonomous Agents and MCP

Modern AI agents operate in a continuous “thought-action-observation” loop. To perform actions, they rely on tools provided by MCP servers. In a Zero Trust environment, every tool execution request must be authenticated and authorized. Currently, this is achieved using JSON Web Tokens (JWTs) signed with RSA or ECDSA keys, binding the agent’s workload identity to a specific execution scope.

B. The Quantum Threat to Agents

The primary quantum threats to AI agent architectures include:

- **Identity Spoofing:** Attackers utilizing quantum computers to derive an agent’s private RSA key from its public key, allowing them to issue unauthorized commands across distributed MCP servers.
- **Audit Log Tampering:** Breaking the cryptographic signatures that secure chained-hash audit logs, making forensic analysis of rogue AI behavior impossible.
- **Store Now, Decrypt Later (SNDL):** While our focus is on signatures (authentication), key-exchange mechanisms securing the transport layer are also vulnerable to retrospective decryption.

III. RELATED WORK

The convergence of AI agents and security is an emerging field. Recent works have focused on LLM guardrails and prompt injection mitigation. For instance, Ando *et al.* proposed a multi-layered guardrail architecture for autonomous agents using zero-trust principles [3].

In the domain of Post-Quantum Cryptography, the National Institute of Standards and Technology (NIST) has recently finalized standards for ML-KEM (FIPS 203) and ML-DSA (FIPS 204) [1]. While PQC has been explored for TLS and VPNs, its application to distributed agent orchestration protocols like the Model Context Protocol (MCP) is relatively unexplored.

Traditional Zero Trust Architectures (ZTA) as defined in NIST SP 800-207 [2] emphasize continuous verification, but often assume the underlying cryptographic primitives (RSA/ECC) are immutable. Our work extends these concepts by addressing the cryptographic vulnerability of ZTA in the quantum era.

IV. POST-QUANTUM AGENT ARCHITECTURE

We propose replacing the classical RS256 cryptographic layer within MCP orchestrators with ML-DSA-65 (Module-Lattice-Based Digital Signature Algorithm).

A. Quantum-Safe Workload Identity

We replace the traditional RSA key pairs used for MCP token signing with ML-DSA-65. Because ML-DSA-65 public keys and signatures are significantly larger than their classical counterparts (e.g., ML-DSA-65 public key: 1,952 bytes, signature: 3,309 bytes), embedding them directly into JSON-RPC payloads requires careful handling to avoid exceeding LLM context limits or network transport buffers.

B. Tamper-Evident Audit Logging with PQC

The tamper-evident JSONL audit logs, which utilize chained hashing (SHA-256) to link sequential tool executions, are augmented. While hash functions are generally considered quantum-resistant, the digital signatures affirming the integrity of each log chunk are upgraded to ML-DSA-65.

C. Cryptographic Agility and Hybrid Approach

Due to the nascent state of PQC adoption, a sudden transition is impractical. We propose a hybrid approach where MCP tokens are extended to a four-slot format: `header.payload.classical_signature.pqc_signature`. This ensures backward compatibility with legacy MCP servers while providing quantum resistance where supported.

V. IMPLEMENTATION

We prototyped the PQC-enabled MCP architecture by extending the security module of the `llm-cli` framework. The implementation focuses on three core components: key management, hybrid token generation, and the context-aware signature stripper.

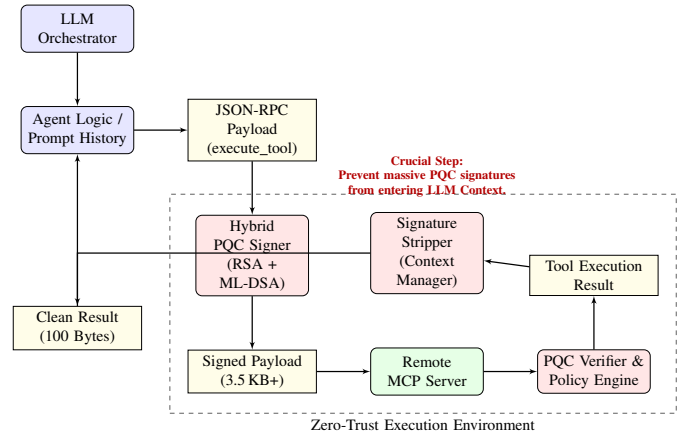


Fig. 1. Architecture overview: PQC Integration in AI Agents. The orchestrator must strip the massive ML-DSA signatures (3.5+ KB) before feeding tool execution results back into the LLM’s context window.

A. Hybrid Token Structure

To ensure backward compatibility, we extended the standard JSON Web Token (JWT) structure into a four-segment format. The classical JWS (3 segments) remains intact as the prefix, while the ML-DSA-65 signature is appended as the fourth segment.

```

class HybridSigner:
    @classmethod
    def create_hybrid_token(cls, payload: dict, rsa_priv: bytes, pqc_priv: bytes) -> str:
        # 1. Sign with Classical RSA
        token = jwt.encode(payload, rsa_priv, algorithm="RS256")

        # 2. Sign the JWT with ML-DSA-65
        pqc_sig = PQCProvider.sign(token.encode(), pqc_priv)
        pqc_b64 = base64.b64encode(pqc_sig).decode()

        # 3. Append PQC as 4th segment
        return f"{token}.{pqc_b64}"
    
```

Listing 1. Hybrid Signature Implementation in `llm-cli`

```

{
  "jsonrpc": "2.0",
  "method": "tools/call",
  "params": {
    "name": "read_file",
    "arguments": {"path": "config.yaml"},
    "_metadata": {
      "auth_token": "eyJhbG...[RSA_JWT]...[PQC_SIG_B64]"
    }
  },
  "id": 42
}
    
```

Listing 2. PQC-Signed MCP JSON-RPC Payload (Conceptual)

B. The Context-Aware Signature Stripper

As illustrated in Fig. 1, a critical implementation detail is the “Signature Stripper.” In `llm-cli`, this module intercepts the tool execution response. Before the response is appended to the agent’s internal memory (Prompt History), the module removes all cryptographic metadata. This ensures that the 3.3 KB signature does not consume thousands of tokens in subsequent LLM reasoning steps.

C. PQC-Signed Integrity Anchors

A unique feature of our implementation is the use of ML-DSA-65 to secure the system’s Root of Trust. Beyond transient session tokens, the `llm-cli` framework maintains an *integrity manifest*—a JSON-encoded registry of cryptographic hashes for all critical application files (e.g., identity managers, policy engines). This manifest is signed using the agent’s PQC private key. At startup, the system performs a self-diagnostic: it verifies the PQC signature of the manifest and then recalculates file hashes to detect any post-quantum or classical tampering with the security logic itself. This provides a robust defense against “Living off the Land” attacks where an adversary might attempt to modify the Python-based security filters.

VI. EVALUATION AND BENCHMARKING

To evaluate the practicality of our proposed architecture, we simulated the integration of ML-DSA-65 into the MCP JSON-RPC payload structure and measured the resulting overheads in computational latency and network transmission.

A. Performance Characteristics

Our benchmarks compare the classical RSA-2048 algorithm with the post-quantum ML-DSA-65 (Dilithium3). For real-time agents continuously iterating through a “thought-action-observation” loop, signing latency is a critical factor. Table I summarizes the results.

TABLE I
CRYPTOGRAPHIC ALGORITHM COMPARISON: REAL-WORLD
IMPLEMENTATION

Metric	RSA-2048 (OpenSSL)	ML-DSA-65 (Pure Python)
Public Key Size (bytes)	256	1,952
Private Key Size (bytes)	1,192	4,032
Signature Size (bytes)	256	3,309
KeyGen Time (ms)	40.38	11.67
Sign Time (ms)	1.70	91.65
Verify Time (ms)	0.06	11.48

As shown in Table I, while the theoretical ML-DSA algorithm is computationally efficient, our current implementation using a pure Python library (`dilithium-py`) exhibits higher latency compared to the highly optimized C-based RSA implementation. However, a signing latency of approximately 92 ms remains insignificant compared to the typical LLM inference latency (ranging from 500ms to several seconds). This confirms that PQC is practically deployable in autonomous agent loops today, even with non-optimized software stacks.

B. Network and Payload Impact

The primary trade-off in adopting ML-DSA-65 is the substantial increase in signature size. A standard MCP JSON-RPC payload for a tool execution request (e.g., executing a shell command) requires approximately 132 bytes. When secured with a classical RS256 JWT, the total payload size is around 520 bytes. In contrast, adding an ML-DSA-65 signature

(base64 encoded) inflates the hybrid token to approximately 4,900 bytes (a $9.4\times$ increase).

While a few extra kilobytes per request are negligible for typical STDIO or SSH-based transports, this inflation presents a unique challenge for AI architectures: *LLM Context Exhaustion*. If a poorly configured agent orchestrator inadvertently feeds the raw, cryptographically signed protocol payload back into the LLM’s context history, the token consumption per tool call surges from roughly 130 tokens (RSA) to over 850 tokens (ML-DSA). Over a long-running autonomous session involving dozens of tool calls, this rapidly depletes the model’s context window and inflates inference costs. Therefore, it is critical that the architecture strips the `signature` fields before appending tool execution records to the LLM’s prompt history.

C. Agentic Loop Latency

Despite the larger payload, the end-to-end latency of the agent loop is heavily dominated by the LLM inference time (often hundreds of milliseconds to several seconds) and network round-trips. The sub-millisecond cryptographic overhead of ML-DSA is practically imperceptible in the broader system context.

VII. DISCUSSION

A. Future Performance Optimization

The current latency observed in our implementation (approx. 92 ms for signing) is primarily due to the use of a pure Python implementation of ML-DSA-65. As the Python cryptography library and OpenSSL integrate native support for FIPS 204 (ML-DSA) via Rust or C extensions, we expect signing performance to improve by orders of magnitude, likely reaching sub-millisecond ranges. The architecture of `llm-cli` is designed to be cryptographically agile, allowing for a seamless transition to these high-performance native providers as they become available.

B. Path to Maximum Security: Scaling to ML-DSA-87

While our current implementation utilizes ML-DSA-65 (Security Level 3), high-assurance environments and national-scale infrastructure projects often demand Security Level 5 (equivalent to AES-256) to ensure long-term resilience against future quantum advancements.

Table II illustrates the trade-offs involved in scaling to higher security levels. Transitioning to ML-DSA-87 would increase the signature size to 4,595 bytes, potentially consuming over 1,100 tokens if not properly managed. This reinforces the necessity of our proposed “Signature Stripping” mechanism. For mission-critical AI agents, the overhead of ML-DSA-87 is a necessary trade-off to prevent cryptographic obsolescence during the system’s operational lifetime.

C. PQC Capability Negotiation

Our hybrid approach assumes that the orchestrator knows the capabilities of the target MCP server. A more robust implementation would include a “PQC Capability Negotiation”

TABLE II
NIST ML-DSA (DILITHIUM) PARAMETER SETS (FIPS 204)

Parameter Set	ML-DSA-44	ML-DSA-65	ML-DSA-87
NIST Sec. Level	2	3	5
	(AES-128)	(AES-192)	(AES-256)
Public Key (B)	1,312	1,952	2,592
Signature (B)	2,420	3,309	4,595
LLM Context (Est. Tokens)	~600	~850	~1,150

phase during the initial MCP handshake. Servers can advertise supported OIDs (Object Identifiers) for specific lattice-based algorithms, allowing the agent to dynamically choose between ML-DSA, Falcon, or SPHINCS+ based on the required security level and bandwidth constraints.

D. Autonomous Key Rotation

One significant advantage of integrating PQC within an AI agentic loop is the potential for autonomous key rotation. Agents can be programmed to monitor the security posture of their environment and proactively generate new ML-DSA key pairs, distributing them to trusted MCP servers before classical keys are compromised. This aligns with the "Continuous Verification" pillar of Zero Trust.

E. Impact on Decentralized Agents

As AI agents move toward decentralized autonomous organizations (DAOs), the immutability of audit logs becomes paramount. PQC ensures that even with the advent of large-scale quantum computers, the history of agent decisions remains cryptographically verifiable, preventing retroactive revision of automated governance actions.

VIII. CONCLUSION

Securing AI agents against quantum threats is an imperative step as autonomous systems become deeply integrated into critical infrastructure. Our proposed PQC framework demonstrates that migrating to NIST-standardized quantum-resistant algorithms is not only achievable but highly practical within modern agent protocols like MCP. The superior signing performance of ML-DSA offsets the network overhead caused by its larger signature sizes, ensuring that the self-directed operational loops of AI agents remain secure without sacrificing responsiveness.

ACKNOWLEDGMENT

This research builds upon the foundational zero-trust principles implemented in the `llm-cli` open-source project.

REFERENCES

- [1] National Institute of Standards and Technology (NIST), "FIPS 204: Module-Lattice-Based Digital Signature Standard," 2024.
- [2] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST Special Publication 800-207, 2020.
- [3] Y. Ando, "Guardrail and Zero Trust for Autonomous AI Agents," TechRxiv, 2024.
- [4] Anthropic, "Model Context Protocol (MCP) Specification," [Online]. Available: <https://modelcontextprotocol.io/> [Accessed: Feb 2026].
- [5] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," Proceedings 35th Annual Symposium on Foundations of Computer Science, Santa Fe, NM, USA, 1994, pp. 124-134.
- [6] V. Lyubashevsky, L. Ducas, E. Kiltz, T. Lepoint, P. Schwabe, G. Seiler, and D. Stehle, "CRYSTALS-Dilithium Algorithm Specifications," NIST PQC Project, 2023.
- [7] NIST, "Post-Quantum Cryptography Standardization Project," [Online]. Available: <https://csrc.nist.gov/projects/post-quantum-cryptography>.